

SEMESTER 1 REPORT
RESEARCH MASTER'S PROJECT

Thesis Proposal

**Cross-Embodiment Generalization and
Efficient Inference Techniques for
Human-in-the-Loop Robot Reinforcement
Learning**

Author: Kartik Dangi

Subject system: `hil_serl_lite` (package `hil_lite`), a from-scratch human-in-the-loop robot reinforcement-learning platform

Report period: Semester 1

Abstract

Human-in-the-loop reinforcement learning lets a robot learn a contact-rich manipulation task from a small number of interventions rather than a large offline dataset, but a research implementation is typically built once, against one specific robot arm, leaving open whether the same code generalises to a structurally different arm without per-arm special-casing, and typically pays whatever inference cost its policy architecture happens to demand without asking whether cheaper alternatives exist. This report covers a semester of work on `hil_lite`, a from-scratch reimplement of HIL-SERL built around RLPD-style sample-efficient off-policy learning, addressing both questions directly. Six structurally heterogeneous robot arms, spanning three different joint-naming conventions and no shared kinematic layout, are registered through one data-driven specification rather than per-arm code branches, and every arm’s workspace is validated by a scripted reachability probe against the real simulated dynamics rather than accepted from a manufacturer datasheet; that validation step caught a genuine defect on one arm; a manufacturer-default configuration converged to within 5 mm of the target position but failed every grasp attempt due to a roll misalignment traced to a near-singular direction in the arm’s own null-space resolution, fixed by retuning a single joint and confirmed by a grasp that now registers sustained, symmetric finger contact forces. Two efficiency techniques developed for large-language-model inference, speculative decoding and consistency distillation, are adapted to a flow-matching manipulation policy and measured rather than assumed to transfer: consistency distillation collapses a 128-forward-pass policy evaluation to a single pass, a measured 7.74 times latency reduction, at a small-teacher-scale fidelity cost this report states plainly rather than rounds up. A third contribution addresses training stability directly: a two-stage manipulation task’s success rate was found to collapse from 70 % to 0 % midway through a single training run and never recover, a regression invisible to every standard training metric and caught only by a purpose-built per-stage diagnostic, root-caused to the composed episode structure itself and resolved by training each stage separately and composing the result, restoring 100 % success with no regression. A fourth strand of work, carried out after the three above and held to the same evidentiary standard rather than a looser one, tests the reimplemented algorithm against an external, third-party benchmark instead of only this project’s own tasks: trained from scratch on a

LIBERO manipulation task with no architecture change beyond what the third contribution already uses, a flow-matching policy over chunked actions reaches 86.7% deterministic success from 0%, a real transfer result rather than an assumed one. Two further refinements adapted from literature published after this report’s core investigation, a multi-horizon critic target and a sigmoid-bounded entropy term, each targeting a training pathology this report’s own third contribution had already characterised, are implemented, correctness-verified in isolation and by direct gradient inspection, and reported honestly as built and verified rather than validated for benefit, since neither has yet been measured head-to-head against its own unmodified baseline. The four contributions are: a cross-embodiment registration architecture whose validation methodology is itself part of the contribution; a measured, honestly-qualified transfer study of two inference-acceleration techniques from language models to robot control; the diagnosis and fix of a non-monotonic training failure invisible to conventional monitoring; and a benchmark-transfer result paired with two literature-adapted refinements, reported under two explicitly different evidentiary standards rather than one blurred together. The underlying reinforcement-learning algorithm itself, SAC with RLPD’s replay-buffer treatment, is a faithful reimplementation, not a novel contribution of this report.

Contents

Abstract	ii
Table of Contents	iii
Nomenclature	v
List of Figures	vi
List of Tables	vii
1 Introduction	2
1.1 Motivation and Scope	2
1.2 Research Questions and Contributions	3
1.3 Report Structure	4
2 Background and Related Work	6
2.1 HIL-SERL and RLPD	6
2.2 Cross-Embodiment Generalisation	7
2.3 Inference Acceleration from Language Models	8
2.4 Hierarchical Structuring of Policies	8
2.5 External Benchmark Transfer and Recent Algorithmic Refinements	9
3 System Design	11
3.1 Simulation Environment	11
3.2 Tasks as Composable Stage Specifications	12
3.3 The Cross-Embodiment Arm Registry	12
3.4 The LIBERO Benchmark Environment	13
4 Software Architecture and Algorithms	16
4.1 Reachability Validation	16
4.2 Consistency Distillation	17
4.3 Speculative Action Verification	18
4.4 Per-Stage Skill Chaining	19

4.5	Q-Chunking	20
4.6	A Multi-Horizon Critic Target for Q-Chunking	20
4.7	Sigmoid-Bounded Entropy	22
5	Evaluation	24
5.1	Evaluation Methodology	24
5.2	Cross-Embodiment Validation (C1)	25
5.3	Inference-Acceleration Transfer Study (C2)	26
5.4	Training-Stability Diagnosis and Fix (C3)	28
5.5	External Benchmark Transfer and Two Algorithmic Refinements (C4) .	31
6	Conclusion and Future Work	34
6.1	Contributions Against Research Questions	34
6.2	Limitations	35
6.3	Future Work	37
6.4	Closing Remarks	38
	References	39
A	Asset Checklist	41
A.1	Generated This Session, From Real Data	41
A.2	Tables	42
A.3	Not Yet Needed	42

Nomenclature

α	Entropy temperature coefficient
ΔQ	Draft/verifier critic disagreement on the draft action
γ	Discount factor
$\mathbf{m}_k$	Zero-padding mask for chunk positions $\geq k$
σ	Logistic sigmoid function
τ	Sigmoid temperature
a_{dim}	Per-step action dimension
c	Sigmoid centring constant
d	SAC actor’s own action dimension
f_ϕ	Consistency network’s learned correction term
g	Distilled consistency function, $g(s, x_t, t)$
$G^{(k)}$	Multi-horizon target for prefix length k
h	Action-chunk length
h_{max}	Per-dimension entropy bonus ceiling
H_{sig}	Sigmoid-bounded entropy bonus
$Q_{\text{draft}}, Q_{\text{verify}}$	Draft and verifier critics’ value estimate of the draft action
Q_{θ^-}	Target critic parameters
$R^{(k)}$	Real discounted k -step partial return
s	Observation passed to the policy network
t	Trajectory position, $t = 0$ noise to $t = 1$ clean action
x_t	Point on the flow/consistency trajectory at position t

List of Figures

3.1	One TaskExpert-solved <code>pick_and_place</code> episode	12
3.2	The six-arm registry and its consumers	14
4.1	Stage transitions at training time and frozen-actor dispatch at evaluation time	19
5.1	All six registered arms, same task scene, same camera	26
5.2	The Sawyer fix, before and after, same real grasp test	27
5.3	Consistency-distillation latency and fidelity	28
5.4	The <code>pick_and_place</code> regression and its fix	29
5.5	LIBERO <code>put_the_bowl_on_the_plate</code> transfer result	32

List of Tables

5.1	Six-arm reachability validation outcome	25
A.1	Figure provenance	41

Chapter 1

Introduction

Human-in-the-loop reinforcement learning lets a robot learn a contact-rich manipulation skill from a small number of live corrections rather than a large pre-collected dataset, closing the gap between a policy that has seen enough demonstrations to imitate and one that has practised enough to recover from its own mistakes. This report covers a semester of work on `hil_lite`, a from-scratch reimplementation of that recipe, built around two questions a single-robot research prototype rarely has to answer: whether the same implementation actually generalises to a structurally different robot arm without being rebuilt for it, and whether a policy architecture’s inference cost is a fixed price of doing business or something that can be measured, challenged, and in one case paid down by a factor of eight.

1.1 Motivation and Scope

The reinforcement-learning algorithm at the centre of this system, soft actor-critic with RLPD’s demonstration-anchored replay treatment [1, 2], and the human-in-the-loop training loop built around it, following HIL-SERL [3], are both reimplemented faithfully in this repository, not reinvented. That distinction is stated here rather than left implicit: this report’s contributions are the cross-embodiment registration architecture, the efficiency-technique transfer study, and the training-stability diagnosis described below, not the underlying learning algorithm, which Chapter 2 credits to its original authors by name.

The repository this report is drawn from is substantially larger than what this report covers. Its own feature matrix lists more than twenty independently switchable research extensions, several explicitly marked as an architecture prototype not yet wired into a trainer, and this report does not claim any of those as a completed contribution. Where a technique this report does cover, consistency-distilled inference or speculative action verification, shares infrastructure with a prototype-stage feature, that boundary is stated at the point it matters, not glossed over. That switchability is

enforced, not merely offered: this repository’s own configuration object raises a named, specific error for an unsupported combination of those extensions’ flags at construction time, more than seventy such interdependent checks in total, rather than letting an invalid combination fail silently or only much later inside a training run. Every number in this report is drawn directly from this repository’s own code, its 68-file, 530-function test suite, or its research documentation (`docs/research/TRAINING_FINDINGS.md`, `docs/research/ROADMAP.md`), and cross-checked against that documentation wherever both exist for the same claim.

1.2 Research Questions and Contributions

Four research questions structure this investigation:

- RQ1** Can one reinforcement-learning implementation generalise across structurally heterogeneous robot arms, different joint counts, naming conventions and kinematic layouts, without per-arm special-casing, and how should reachability be validated rather than assumed from a manufacturer specification?
- RQ2** Which efficiency techniques developed for large-language-model inference, speculative decoding and consistency distillation, transfer meaningfully to closed-loop robot control, and by how much, when applied to a flow-matching policy?
- RQ3** How does per-stage skill-chaining affect training stability on a multi-stage manipulation task relative to a single monolithic policy?
- RQ4** Does the reimplemented algorithm transfer to an external, third-party manipulation benchmark without architectural change, and can literature published after this report’s core investigation, targeting the specific training pathology RQ3’s own diagnosis already characterised, be adapted and correctness-verified ahead of a full head-to-head evaluation?

Four contributions address these questions:

- C1 (RQ1)** A six-arm registration architecture (Panda, UR5e, UR10e, xArm7, FR3, Sawyer), each arm’s joint names, kinematics and workspace held as data rather than encoded in per-arm branches, validated by a scripted reachability probe against the real simulated dynamics and real grasp attempts rather than accepted from each arm’s manufacturer configuration. That validation step is reported not only as a methodology but through the specific defect it actually caught: a Sawyer configuration that reached its target position to within 5 mm and still failed every grasp, traced to a near-singular direction in the arm’s own redundant null

space, is documented as a worked example of why the validation step matters, in Section 5.2.

- C2 (RQ2)** A measured transfer study of two large-language-model inference techniques applied to a flow-matching manipulation policy: consistency distillation, collapsing a 128-step policy evaluation to a single forward pass for a measured 7.74 times latency reduction, and speculative action verification, a cheap policy proposing an action that an expensive critic either accepts or overrides. Both are reported with the honest limitation the measurement itself surfaced, not rounded up to an unqualified success: the distilled policy’s fidelity to its teacher’s action distribution is measurably poor at the small scale this was trained and tested at.
- C3 (RQ3)** The root-caused diagnosis of a non-monotonic training regression on a two-stage manipulation task, a stage success rate collapsing from 70 % to 0 % and never recovering across the remaining two-thirds of a training run, while every standard training metric, including the episode-level success rate logged during training itself, stayed in its normal range throughout. The fix, training each stage separately and composing the frozen result rather than continuing to patch the monolithic run, restored 100 % success with no regression, and is the strongest single result in this repository’s own training record.
- C4 (RQ4)** A benchmark-transfer result and two literature-adapted algorithmic refinements, reported under two explicitly different evidentiary standards rather than blurred into one. Trained from scratch on a task from LIBERO [4], an external, third-party manipulation benchmark this repository’s own tasks played no part in designing, the flow-matching policy class Section 4.2 already introduces, now additionally combined with Q-chunking [5], reaches 86.7 % deterministic success from 0 %, a real, twice-confirmed transfer result. A SEAR-inspired [6] multi-horizon critic target and a SigEnt-SAC-inspired [7] sigmoid-bounded entropy term, each adapted deliberately rather than ported wholesale and each targeting a training pathology this report’s own C3 investigation had already characterised, are implemented and correctness-verified in layers, then reported honestly as built and verified, not validated for benefit: neither has yet been measured head-to-head against its own unmodified baseline, a distinction this report states plainly rather than rounding up to a finished result.

1.3 Report Structure

The remainder of this report is organised as follows:

- **Chapter 2** credits HIL-SERL and RLPD as the algorithm this repository reimplements, and reviews the large-language-model inference techniques, hierarchical reinforcement-learning, Q-chunking and the two further-literature refinements the other three contributions draw on.
- **Chapter 3** describes the simulation environment, the task and stage specification system, the data-driven arm registry C1 is built on, and the external LIBERO environment C4’s benchmark-transfer result is measured against.
- **Chapter 4** presents the reachability-validation methodology, the consistency-distillation and speculative-action mechanisms, the per-stage skill-chaining architecture, and the Q-chunking, multi-horizon-critic and sigmoid-bounded-entropy mechanisms behind C4.
- **Chapter 5** reports the six-arm validation results including the Sawyer worked example, the measured latency and fidelity figures for both inference techniques, the full training-regression-and-fix account, and the LIBERO transfer result alongside the verification account for both C4 refinements.
- **Chapter 6** summarises the four contributions against the four research questions and states which of this repository’s other research extensions remain future work rather than completed contributions.

With the research questions declared and the reimplementation boundary stated, the next chapter credits the algorithm this system builds on before turning to the techniques that are this report’s own contribution.

Chapter 2

Background and Related Work

This chapter credits the algorithm this system reimplements before reviewing the two further bodies of work its contributions draw on: inference-acceleration techniques developed for large language models, and hierarchical structuring of reinforcement-learning policies. It is a briefing rather than a survey, since none of this report’s three contributions proposes a new learning algorithm.

2.1 HIL-SERL and RLPD

The training loop this repository implements follows HIL-SERL [3] directly: a human operator supplies live corrective interventions during autonomous rollouts rather than a fixed offline demonstration set, and those interventions are routed into the replay buffer alongside autonomous experience so the policy is trained on a mixture of expert-corrected and self-generated transitions throughout. The off-policy learning algorithm underneath is soft actor-critic [1], maximum-entropy actor-critic learning with a stochastic policy, and the specific treatment of demonstration data follows RLPD [2]: demonstrations and online experience are kept in two separate buffers and sampled in a fixed ratio per batch, decayed over training rather than mixed once and left static, so the policy leans on demonstrations early and increasingly on its own experience as training progresses. This repository reimplements both from scratch rather than importing either reference implementation; the learning algorithm itself is not this report’s contribution, and Section 1.1 states that distinction directly rather than leaving it to be inferred.

That reimplementation is a claim this report can point to specific code for, not one taken on faith. The tanh-squashed Gaussian policy’s log-probability carries an explicit Jacobian correction term, easy to omit without the resulting policy appearing obviously broken, written directly into `sac.py`’s own `sample_and_log_prob` rather than delegated to a probability-distribution library that might supply it silently or not at all. The twin critic heads, `DoubleCritic` in the same file, share a single observation

encoder rather than each carrying an independent copy, halving the encoder’s own parameter and activation cost with no change to the twin-critic estimate itself. A bootstrap mask, `masks = 1 - terminated` at the point the temporal-difference target is built, distinguishes an episode ending in genuine termination from one ending only because a time-step limit was reached, a well-known, easy-to-introduce error in reinforcement-learning implementations generally, since the two cases require different treatment in that target and getting this wrong rarely makes training visibly diverge, only quietly biases it. None of this is this report’s own contribution either, but it is the concrete substance the phrase “reimplemented faithfully” refers to, rather than an assertion the reader is asked to accept unexamined.

Why the correction loop is needed at all, not only offered as an option, is also measured directly in this repository rather than argued from the general covariate-shift case for interactive learning. A tanh-Gaussian actor cloned by supervised regression on sixty scripted demonstrations, with no correction of any kind, reaches 55 % success over twenty evaluation episodes on the two-stage `pick_and_place` task, the ceiling this repository’s own `train_bc.py` plateaus at before RLPD fine-tuning is applied: a concrete instance of the gap between a policy that has seen enough demonstrations to imitate and one that has practised enough to recover from its own mistakes, the distinction Section 1.1 opens this report by naming. That ceiling is only trustworthy because the cloning objective is numerically sound in the first place: learning the policy’s variance jointly with its mean is unstable on the near-deterministic, saturating actions a scripted or teleoperated demonstrator produces, the variance collapses and any residual error then explodes the loss by several orders of magnitude, so `train_bc.py` instead holds the variance fixed and trains the mean alone by direct regression, adapting the variance only later, during SAC fine-tuning.

2.2 Cross-Embodiment Generalisation

Training one policy, or one implementation, across structurally different robots rather than one per platform has become an active research direction as manipulation datasets have grown large enough to span multiple robot embodiments at once, most visibly in large multi-institution efforts pooling demonstration data across dozens of distinct robot platforms to train a single generalist policy. The cross-embodiment problem this report addresses is narrower and sits one level below that: not a single policy trained jointly across robots, but a single *implementation*, the same environment code, task specification and training loop, applicable to a structurally different robot arm without per-arm branches in the code itself. This narrower problem still requires confronting the same underlying fact that motivates the broader one: different robot arms do not share a naming convention, a kinematic layout, or even a joint count, so any claim of

generalisation has to be checked against the specific arm’s own geometry rather than assumed from family resemblance to an arm that already works. Chapter 3 describes how this repository holds that geometry as data, and Chapter 4 describes how it is validated rather than trusted once entered.

2.3 Inference Acceleration from Language Models

Two techniques this report transfers to robot control were developed to address a cost that is structurally similar in both domains: an autoregressive or iterative model that is accurate but expensive to sample from at every control step.

Speculative decoding. In large-language-model inference, speculative decoding [8] uses a small, cheap draft model to propose several tokens ahead, then verifies that proposal against the large target model in a single batched forward pass, accepting the draft wherever the two models agree and falling back to the expensive model only where they diverge. The technique is exact, not an approximation, because the final accepted output is always what the expensive model would have produced alone; the saving comes from how rarely the expensive model has to be run token by token. Section 4.3 describes how this repository maps the same draft-verify structure onto a single robot action rather than a sequence of tokens.

Consistency distillation. Diffusion and flow-based generative models produce a sample through many small iterative refinement steps, each a full forward pass through the underlying network. Consistency models [9] are trained to map any point on that same iterative trajectory directly to the trajectory’s clean endpoint in a single forward pass, distilled from an already-trained iterative teacher rather than trained from scratch. Applied to a flow-matching policy [10], the technique promises to collapse a many-step action-sampling procedure into one; whether that promise holds in practice, and at what cost to the sampled action’s fidelity to the teacher’s own distribution, is an empirical question this report answers directly rather than assumes, in Section 5.3.

2.4 Hierarchical Structuring of Policies

Decomposing a long-horizon task into a sequence of shorter sub-goals, each solved by its own policy, is a long-standing idea in reinforcement learning, formalised in the options framework [11] as temporally extended actions that can be composed the same way primitive actions are. The specific instance this report evaluates, training each stage of a two-stage manipulation task as its own short-horizon problem and composing the frozen result rather than training one policy across the full composed episode, is a

direct application of that decomposition principle, motivated in this repository’s own training record by a concrete failure the monolithic alternative produced, detailed in Section 5.4 rather than assumed from the general argument for decomposition alone.

2.5 External Benchmark Transfer and Recent Algorithmic Refinements

The three research questions above are all answered against tasks this repository itself defines. A fourth question, addressed after that investigation was complete, asks a different thing: whether the reimplemented algorithm transfers to a task nobody in this project designed, and whether literature published after the investigation offers a correctness-verifiable improvement to a pathology it had already found.

LIBERO and Q-chunking. LIBERO [4] is a third-party benchmark suite of robot manipulation tasks built to study knowledge transfer across a robot’s lifetime, structured into task suites spanning object, spatial, goal and long-horizon variation. The task this report trains against, `put_the_bowl_on_the_plate` from its `libero_goal` suite, is one this repository’s own task-composition system (Section 3.2) played no part in designing, run through a cross-process bridge to LIBERO’s own real simulator rather than reimplemented in this repository’s own scene format. Q-chunking [5] restructures off-policy reinforcement learning to act over an h -step action chunk rather than a single action per decision, so a temporal-difference backup spans h environment steps at once (discount γ^h) rather than one, motivated by the same long-horizon, sparse-reward setting the LIBERO task above sits in. Section 4.2’s flow-matching policy is the actor this report combines it with, described in full in Section 4.5.

Two further-literature refinements targeting an already-diagnosed pathology. Section 5.4’s training-record investigation characterises a mechanism this report calls the alpha-gated plateau: a critic that settles at a fixed point pricing only its own policy’s entropy bonus rather than task value, until the entropy temperature anneals low enough for a sparse task reward to stand out above that floor. Two papers published after that investigation was complete diagnose closely related problems from first principles rather than in response to this repository’s own finding, independent corroboration rather than a fix built to order. SEAR [6] diagnoses naive action-chunked off-policy learning as under-supervised, exactly one temporal-difference target per whole chunk regardless of chunk length, and proposes a multi-horizon critic target as the fix. SigEnt-SAC [7] diagnoses the standard maximum-entropy bonus itself, an unbounded, sign-indefinite log-density term, as capable of driving exactly the plateau this report’s

own training record already found, and proposes a bounded replacement. Both diagnoses are checked against this repository’s own code, not accepted from the abstract alone, and both fixes are adapted rather than ported wholesale, in Sections 4.6 and 4.7.

With the algorithmic and technique-level background in place, the next chapter turns to how this repository’s own architecture puts each of these ideas into code.

Chapter 3

System Design

This chapter describes the simulation environment, the task and stage specification system built on top of it, and the data-driven arm registry Chapter 4’s cross-embodiment validation methodology operates on, corresponding to `hil_lite/envs/` and `hil_lite/envs/robots/arms/` in the `hil_serl_lite` repository.

3.1 Simulation Environment

Every task runs in MuJoCo [12], built around gym-hil’s Franka Panda model (physics, meshes and keyboard or gamepad teleoperation reused directly) plus a scene compiler that assembles task-specific objects, cubes and target zones, around that base model at load time from a task specification rather than a separate hand-built scene file per task. Two observation modes are supported throughout the codebase: a state mode exposing the simulator’s own ground-truth object and proprioceptive state directly, used for fast iteration and for the scripted expert described below, and an image mode rendering the simulator’s cameras for a vision-based policy, the mode a real deployment would actually use since ground-truth object pose is not normally available outside simulation.

A task-generic scripted expert, **TaskExpert**, solves every shipped task from ground-truth state, including the three-stage stack task, and stands in for a human operator in every pipeline stage that would otherwise need one: collecting demonstrations, playing the intervening corrective role during training, and driving the earlier stages of a composed task while a later stage is trained in isolation (Section 4.4). This is what makes every result in this report reproducible headless, without a display or a human at the controls, and is stated here because it recurs as a load-bearing mechanism in more than one later chapter, not only as a convenience feature.

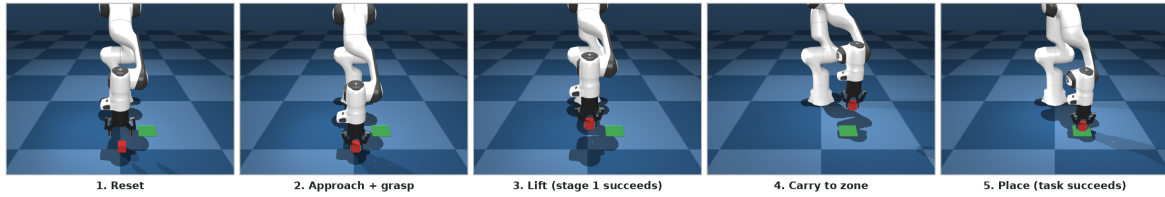


Figure 3.1: One real, unstaged episode of `pick_and_place` solved by `TaskExpert` on the Panda arm, rendered directly from this repository’s own simulator. The stage boundary is not a visual guess: frame 3 is the exact simulation step at which the *lift* stage’s own predicate first evaluates true, and frame 5 is the step at which the *place* stage’s predicate does, ending the episode successfully. Generated by `scripts/figures/fig_task_sequence.py`, which asserts the logged episode actually succeeds before writing the figure.

3.2 Tasks as Composable Stage Specifications

One YAML file defines one task: the scene objects it needs, an ordered list of stages, each a named sub-goal predicate such as “cube lifted” or “cube in the target zone”, and the pipeline of steps, demonstration collection, training, evaluation, that task runs through. Tasks compose: a two-stage task can be declared as the composition of two existing single-stage tasks rather than redefined from scratch, with shared scene objects (a cube appearing in both parent tasks) merged into one rather than duplicated, and stages running in the declared composition order. The two-stage `pick_and_place` task, composed from `lift_cube` and `place_in_zone`, is the task Chapter 5’s training-stability result (Section 5.4) is built around; its own specification declares a 200-step episode budget, a 60 000-step training budget, and a 30-episode scripted demonstration collection step ahead of training, exactly the configuration that result was measured under.

3.3 The Cross-Embodiment Arm Registry

Six robot arms, a Franka Panda, a Universal Robots UR5e, a Universal Robots UR10e, a UFactory xArm7, a Franka FR3 and a Rethink Robotics Sawyer, are registered through one shared `ArmSpec` data structure rather than six parallel code paths. Each entry records the arm’s joint names, its attachment site for the gripper, a home joint configuration, a cartesian workspace bound, the camera names its scene exposes, and where its MJCF model definition is fetched from. Figure 3.2 shows how this single specification feeds every downstream consumer, the generic task environment, the configuration system, and the web dashboard, none of which contain arm-specific branches.

Three design decisions in this structure are worth stating explicitly, since each addresses a real failure mode rather than being a default left unquestioned.

Every joint name is data, confirmed against the real model file. A shared registry is only safe if the assumption it is built on, that per-arm differences can be fully captured as data rather than code, actually holds. It does not hold at the level of naming convention: the Panda’s own MJCF names its joints `joint1` through `joint7`, the UR5e’s names its shoulder joint `shoulder_pan_joint`, and the xArm7’s names its actuators `act1` through `act7` while its joints keep the Panda-style `joint1` through `joint7` pattern independently, three conventions with no shared structure to infer from one arm to the next. Every joint name in the registry is recorded from directly reading the corresponding MJCF file, not inferred from another arm’s convention or the manufacturer’s own naming scheme in general, a distinction that mattered concretely for the FR3 (Section 5.2): despite sharing Franka’s exact kinematic chain with the Panda, its joints are named `fr3_joint1` through `fr3_joint7`, not `joint1` through `joint7`, which a naming convention copied from the Panda entry would have silently gotten wrong.

Model provenance is pinned, not floating. Every arm’s MJCF definition is fetched from Google DeepMind’s MuJoCo Menagerie at one fixed, recorded commit hash rather than the repository’s default branch, so an upstream change to a robot’s model cannot silently change what that robot means to this project mid-project; the pinned commit is bumped deliberately, verified against the upstream repository at bump time, never automatically.

Structural validation is enforced in the data type itself. The `ArmSpec` data-class validates its own internal consistency on construction, that its home configuration supplies exactly one joint angle per declared joint, and that a menagerie source path is never given without the commit it should be pinned to, so a malformed registry entry fails immediately at definition time rather than producing a confusing error deep inside a training run.

Holding this much structural difference as pure data is only trustworthy if the resulting workspace and reachability claims are actually checked against the real simulated arm, not merely against the shape of the data structure; Chapter 4 describes exactly that validation step, and the defect it caught on one of these six arms.

3.4 The LIBERO Benchmark Environment

C4’s benchmark-transfer result (Section 2.5) needs the actual LIBERO [4] simulator running, not a reimplementations of one of its tasks in this repository’s own scene format, since the point of the result is that this repository’s own training loop transfers to a task it played no part in designing. LIBERO’s own dependency, `robosuite==1.4.0`, pins

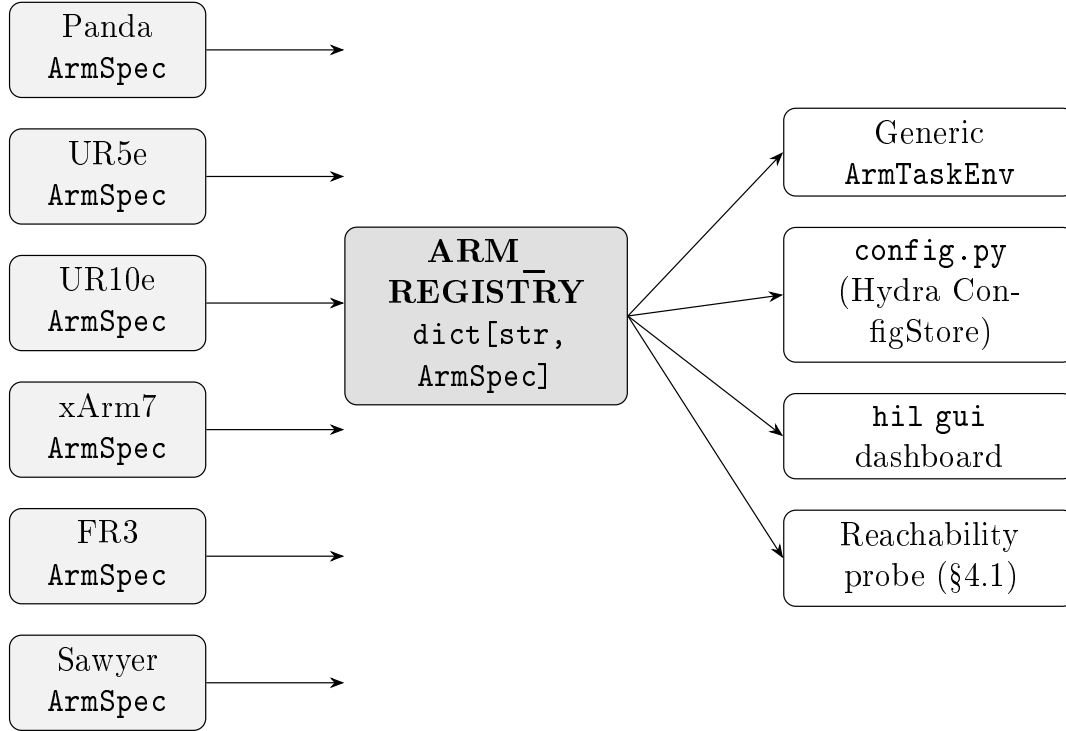


Figure 3.2: One `ArmSpec` per arm, read as data by every downstream consumer. No consumer contains a per-arm code branch; adding a seventh arm means adding a seventh registry entry, not touching the environment, configuration, or dashboard code.

`mujoco~=2.3`, a full major version behind the `mujoco 3.x` this repository’s own arm registry (Section 3.3) and every other environment in this project depend on; installing LIBERO’s real dependencies into the same Python environment as `hil_lite` would silently downgrade `mujoco` for the entire install, a conflict confirmed directly by testing the newer `mujoco` against LIBERO’s own robosuite version and reading the resulting assertion failure, not assumed from the two projects’ listed requirements alone.

Rather than resolve that conflict by forking or patching either dependency, LIBERO is isolated in its own virtual environment and bridged to this repository’s own training process over a local subprocess boundary: a server process (`benchmarks/libero/libero_server.py`) holds the real LIBERO environment and exposes it over a length-prefixed JSON protocol on a local port, and a client-side adapter (`hil_lite/envs/libero_backend.py`) implements this repository’s own environment interface against that protocol, selected by `env_backend=libero` exactly like any other environment backend this project supports. Everything downstream of that interface, the replay buffers, the SAC/RLPD training loop, the flow-matching policy and Q-chunking (Section 4.5), is unmodified: LIBERO is one more environment behind an interface those components already only depend on, not a special case threaded through them.

The specific task trained against, `put_the_bowl_on_the_plate` from LIBERO’s own `libero_goal` suite, is demo-seeded from LIBERO’s own real, pre-recorded hu-

man demonstrations, 49 of the suite’s 50 successfully converted into this repository’s own per-step episode format (4 531 transitions in total) rather than collected fresh, so the demonstration data itself is LIBERO’s own, not this repository’s scripted expert imitating it.

With the arm registry’s validation methodology and the LIBERO bridge both in place, the next chapter turns to how this repository’s own architecture puts each of this report’s mechanisms into code, including the ones C4 is built from.

Chapter 4

Software Architecture and Algorithms

This chapter presents the mechanisms behind the four contributions declared in Chapter 1: how each of the six registered arms is actually validated rather than trusted from its specification, how the two large-language-model inference techniques are adapted to a robot policy, how per-stage skill chaining is composed from independently trained stage policies, and how Q-chunking and its two further-literature refinements extend that same policy to an external benchmark.

4.1 Reachability Validation

Holding an arm’s workspace as a declared bound in its `ArmSpec` (Section 3.3) is only trustworthy if that bound is checked against what the arm can actually reach in simulation, not copied from a manufacturer’s reach specification or another arm’s already-working bound. Every arm’s cartesian workspace bound in this repository is derived from a scripted probe run against the real `ArmTaskEnv` step loop, the identical control path the arm executes at training and evaluation time: from the arm’s reset pose, a small, bounded per-step action drives the end effector toward each of the candidate workspace box’s eight corners in turn, for a fixed number of steps per corner, with every step checked for a non-finite (NaN) state, which would silently indicate a physics or control failure rather than a merely slow approach. A corner is accepted once the end effector settles within a stated tolerance of it; a corner that instead plateaus short of that tolerance, re-run for extra steps to rule out slow convergence and checked against every joint’s own limit to rule out a joint simply running out of range, is treated as a genuine reach limit and the workspace bound is narrowed to exclude it, rather than accepted on the assumption that more simulation time would eventually close the gap. This procedure, and its outcome for each of the six arms, is repeated as a regression-guarded test alongside the registry entry it validates, so a future change to an arm’s model or home configuration that breaks reachability is caught automatically rather than only at the next manual re-check.

The Sawyer worked example. Applying this procedure exposed a defect a specification-only check would have missed entirely. Sawyer’s own manufacturer keyframe, used as the initial home configuration exactly as it was for the Franka FR3, placed the gripper’s target position within 5mm of the task’s cube on reset. Position alone was not sufficient: a scripted approach-and-grasp test against the real cube, the same test every other registered arm is checked against, converged the end effector to the cube’s position but failed to register a grasp on every attempt. The two gripper finger pads settled with a stable 15mm-scale roll misalignment between them, one side measurably higher than the other, so closing the gripper made the finger pads contact each other around the side of the cube rather than clamp it.

Three specific, checkable explanations were ruled out before the actual cause was accepted, rather than the first plausible-sounding explanation being assumed correct: the misalignment held steady across an extended run rather than slowly resolving, ruling out a slow-settling controller artifact; every joint retained comfortable margin from its own range limit, ruling out a joint simply running out of travel; and the commanded joint torque matched what the controller’s own operational-space control law asked for exactly, with no evidence of clipping against any actuator’s torque limit, ruling out actuator saturation. What remained was a genuine near-singular direction specific to roll, a consequence of how this particular seven-degree-of-freedom arm’s redundant null-space resolution behaves when servoing from this exact home configuration to this exact target position, not a resource limit anything in the simulation could simply be given more of.

The fix follows the same pattern already used for the UR5e and UR10e’s own reach-direction issues: the arm’s shoulder-equivalent joint, left at the manufacturer’s default of zero, was retuned iteratively away from that default while every other joint was left exactly as the manufacturer’s keyframe specifies it. A single value, moving that one joint to -0.3rad , resolves the roll misalignment: the identical approach-and-grasp test now registers a sustained grasp with real, comparably sized finger contact forces on both pads, confirmed directly by reading the simulated contact forces rather than inferring success from position alone. Section 5.2 reports this fix alongside the equivalent validation outcome for every other registered arm.

4.2 Consistency Distillation

A flow-matching policy [10] samples an action by numerically integrating a learned velocity field across many small steps, each step a forward pass through the policy network; evaluating the policy with a best-of-several-samples selection multiplies that cost further. Consistency distillation [9] trains a second, smaller network to map any point along that same integration trajectory directly to its clean endpoint in a single

forward pass, learned from an already-trained flow policy acting as a frozen teacher rather than trained from the task reward directly.

The distilled network is skip-parameterised so that the defining boundary condition, that the network’s output at the trajectory’s own endpoint is exactly the input action with no approximation, holds by construction rather than merely being encouraged by the training loss:

$$g(s, x_t, t) = x_t + (1 - t) \cdot f_\phi(s, x_t, t) \quad (4.1)$$

where s is the observation, x_t is the current point on the trajectory, $t \in [0, 1]$ is the trajectory position with $t=1$ the clean-action endpoint, and f_ϕ is the network’s own learned correction term. At $t=1$, the second term vanishes identically and g returns x_t exactly, satisfying the boundary condition $g(s, x, 1) = x$ without relying on the network having learned to do so. The student is trained by self-distillation against its own stop-graduated prediction one teacher integration step earlier, a simplification of the published recipe’s separate exponential-moving-average teacher network, stated in this repository’s own code as a deliberate trade at the network scale this system runs at: a reasonable simplification for demonstrating the mechanism and measuring its inference-cost saving, at the cost of training stability the full published recipe would improve on. At inference time the distilled network replaces the flow policy’s entire multi-step integration with one forward pass, the mechanism Section 5.3’s latency figure is measured against.

4.3 Speculative Action Verification

Speculative decoding’s draft-then-verify structure [8] maps onto a single robot action as follows. A cheap policy, any already-implemented actor with its own learned critic, proposes an action through one inexpensive forward pass. That action is then scored twice: once by the cheap policy’s own critic, and once by a more expensive policy’s critic alone, not yet its more expensive actor. The two scores are compared:

$$\Delta Q = Q_{\text{draft}}(s, a_{\text{draft}}) - Q_{\text{verify}}(s, a_{\text{draft}}) \quad (4.2)$$

where Q_{draft} and Q_{verify} are the two critics’ own minimum-over-twin-critics value estimate of the same state-action pair. If $|\Delta Q|$ stays within a fixed disagreement threshold, the draft action is accepted as-is and the expensive policy’s own actor is never invoked at all; only when the two critics disagree by more than that threshold is the expensive actor called to produce a replacement action. This is the specific property that makes the mechanism speculative rather than simply routing every action through the more expensive model: the common, cheap path never evaluates the expensive actor, only its comparatively cheap critic, so the saving is real whenever the draft policy and the

more expensive verifier already agree.

4.4 Per-Stage Skill Chaining

A multi-stage task’s stages are tracked by a wrapper around the base environment that treats a non-final stage’s own success as a sub-goal rather than the end of the episode: reaching a non-final stage’s success condition advances the active stage and swaps in that stage’s own reward source, without terminating the episode, and the episode-level success flag is explicitly reset to false on that transition so that episode-level logging keeps meaning “the whole task succeeded”, not “some stage did”. Only the final stage’s own success actually ends the episode.

Training time, every episode (`MultiStageWrapper.step`)

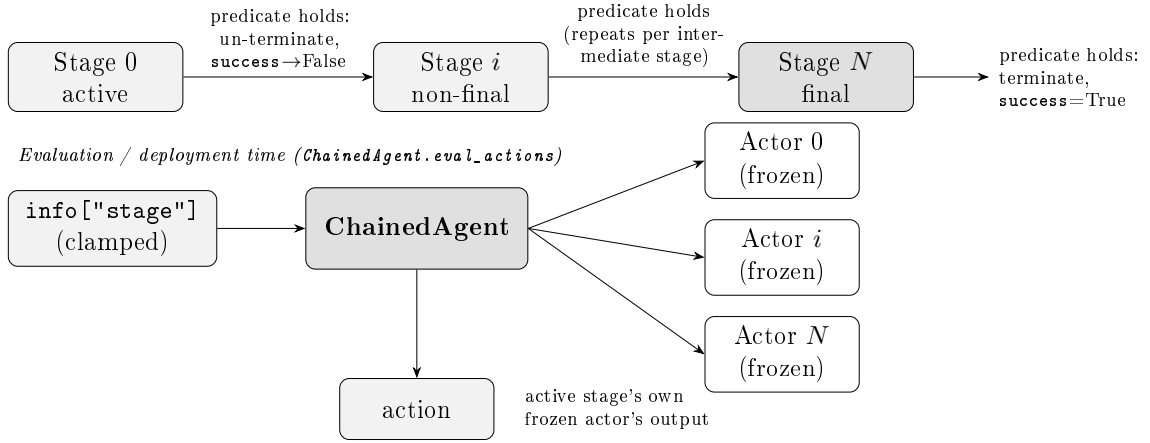


Figure 4.1: Two separate moments this section describes. *Top*: every episode’s stage transitions under `MultiStageWrapper`, whatever agent is training against it. *Bottom*: `ChainedAgent` at evaluation or deployment time, dispatching to one frozen, independently trained actor per stage. The two are drawn separately because chaining composes already-trained actors after the fact; nothing shown in the bottom half runs during any stage’s own training.

Skill chaining, the mechanism Section 5.4 evaluates, trains a separate policy for each stage of a composed task rather than one policy across the full composed episode. Each stage’s own training run uses the scripted expert (Section 3.1) to drive every earlier stage to completion first, so that stage’s training start-states are exactly the hand-off states it will actually see once composed with the stages before it, then trains only that stage as its own short-horizon problem. At evaluation or deployment time, a `ChainedAgent` holds one frozen, independently trained actor per stage and dispatches to the active stage’s own actor based on the stage index the environment wrapper reports, clamped to a valid stage index so an out-of-range report cannot select a non-existent policy. This composition is evaluation-only: nothing is trained inside the chained agent itself, only inside each stage’s own separate training run, which is what keeps

each individual training problem short-horizon and correspondingly easier to fit on the resource-constrained training setup this repository targets. Section 5.4 reports the specific training failure that motivated adopting this composition over a single monolithic policy for the task it is evaluated on, not as a default assumed better in general.

4.5 Q-Chunking

Q-chunking [5] restructures off-policy reinforcement learning to act over an h -step chunk of actions rather than a single action per environment step, adopted for C4 specifically because a temporal-difference backup then spans h environment steps in one update, exactly the property a long-horizon, sparse-reward task such as LIBERO’s `put_the_bowl_on_the_plate` (Section 3.4) rewards. This repository’s own implementation makes two changes to wire chunking through an otherwise unmodified actor and critic, rather than building a chunk-aware network of its own: the agent is constructed with its action dimension widened to h times its per-step value and its discount raised to the h -th power,

$$a'_{\text{dim}} = h \cdot a_{\text{dim}}, \quad \gamma' = \gamma^h \tag{4.3}$$

so neither the flow-matching actor (Section 4.2) nor the twin-critic architecture needs any chunk-specific code; and a per-step demonstration episode is re-viewed as a sliding window of h -step chunks at stride one, reward the discounted in-window sum and the window ending early, mask zero, wherever the underlying episode terminates inside it, so per-step demonstrations already on disk stay usable without being re-collected in chunk-shaped form. This is the actor and critic C4’s LIBERO result is measured with, at $h=4$; Section 4.6’s refinement is an auxiliary term layered on top of this same critic, not a different one.

4.6 A Multi-Horizon Critic Target for Q-Chunking

Q-chunking’s own critic, as built in Section 4.5, gives every h -step chunk exactly one temporal-difference target, at the chunk’s end; SEAR’s own diagnosis (Section 2.5) is that this under-supervises the critic relative to how many actions the chunk actually commits it to. Confirmed against this repository’s own critic-loss function directly, not assumed from the diagnosis alone: `hil_lite/agents/flow.py`’s critic loss computes exactly one temporal-difference target per whole chunk, before this section’s own addition.

The fix adds an auxiliary loss term, for every prefix length $k \in \{1, \dots, h-1\}$ of the chunk, a target bootstrapped from the real state actually reached after k sub-steps

rather than an estimated one:

$$G^{(k)} = R^{(k)} + \gamma^k \cdot \mathbb{I}[\text{valid}_k] \cdot \min_i Q_{\theta^-}^{(i)}(s_{t+k}, a' \odot \mathbf{m}_k) \quad (4.4)$$

where $R^{(k)}$ is the real discounted k -step partial return, s_{t+k} is the real intermediate state reached after k sub-steps, a' is a chunk freshly sampled from the flow actor at s_{t+k} , $\mathbf{m}_k$ zero-pads every action slot at position k and beyond so the critic sees only a k -action commitment, and valid_k is false wherever position k lies past an episode termination, in which case the bootstrap term is masked out and $G^{(k)} = R^{(k)}$ alone. The critic trains against $h-1$ such targets in addition to its existing full-chunk target, each contributing to the loss only for rows whose valid_k mask is set.

What is ported, what is adapted, and what is deliberately not built. SEAR trains a purpose-built causal-transformer critic with a native per-prefix output head. This repository’s own critic (Section 4.5) is a single-output twin-critic network shared with the unchunked case, and rebuilding it as a transformer would touch every checkpoint format and every other configuration that critic is shared with, for a component SEAR’s own ablation states is not what drives its result: replacing their transformer with a plain network loses little, while keeping the transformer but removing the multi-horizon targets loses most of the gain. On that evidence, the target formula in Equation 4.4 is ported, and the transformer is not: the existing critic is instead queried once per prefix length with a zero-padded action, relying on the network to read the padding pattern as an implicit horizon signal. The other half of SEAR’s own mechanism, genuine bootstraps from genuine intermediate states rather than an approximated target, is kept faithfully, but scoped to the demo buffer only: per-step demonstrations already record every intermediate observation, so this repository’s own sliding-window chunk view (Section 4.5) can attach s_{t+k} for free, while the online chunk executor built for the unaugmented case does not retain intermediate observations at all, and teaching it to would be new online-collection machinery, not a critic-loss change. This is a stated scope limit, not an oversight left for a reader to notice independently.

Verification. Layered, cheapest first, following this project’s own standing verification discipline. Unit tests check the partial-return and validity-mask arithmetic against hand-computed values, including a termination-inside-the-window edge case. A JAX-level integration test confirms the auxiliary loss produces finite values and real gradient flow through every code path the training loop actually uses, a single update, a critic-only update, and the fused multiple-update-per-batch path alike, and that a row whose validity mask is unset contributes exactly zero regardless of what its own intermediate data holds, confirming the masking is structural rather than merely usually correct. A

regression test confirms the flag left off is byte-identical to before this section’s own addition: the auxiliary loss is exactly 0.0 and no new batch fields are produced. A real, short end-to-end run against this repository’s own LIBERO demonstrations (Section 3.4), 300 steps, confirms no crash and the auxiliary loss itself falling smoothly, 0.27 to 0.007 over the run, a genuine quantity the critic is visibly learning to reduce, not merely a code path that executes without error. Section 5.5 states plainly what this verification does, and does not, establish.

4.7 Sigmoid-Bounded Entropy

This section’s own refinement targets the same training pathology as Section 4.6’s, but is otherwise independent of it: it applies to this repository’s tanh-Gaussian SAC actor (Chapter 2), not the flow-matching, Q-chunked actor Section 4.6 augments, and the two are not combined in any single run this report describes.

The alpha-gated plateau (Section 5.4) is, in this report’s own terms, a fixed point of the soft Bellman backup: the critic settles at $Q(s, a) \approx \gamma \alpha H / (1 - \gamma)$, pricing only the policy’s own future entropy bonus rather than task value, until the entropy temperature α anneals low enough for a sparse task reward to stand out above that floor. SigEnt-SAC’s own diagnosis, read from its own Equation 1 rather than reconstructed from its abstract, locates a specific mechanical cause available to attack directly: the standard entropy bonus is $-\log \pi(a|s)$, a continuous log-density that is unbounded below and sign-indefinite, so a sufficiently peaked policy scores a large positive log-density at its own sampled actions and the entropy bonus swings deeply negative rather than staying a small, bounded quantity. Their fix replaces each action dimension’s raw log-density with a temperature-controlled sigmoid of its surprisal:

$$H_{\text{sig}}(s, a) = \sum_{i=1}^d h_{\text{max}} \cdot \sigma\left(\frac{s_i - c}{\tau}\right), \quad s_i = -\log \pi_i(a_i | s) \quad (4.5)$$

strictly positive and bounded in $(0, d \cdot h_{\text{max}})$ by construction, where d is the SAC actor’s own action dimension, c is a centring constant and τ a temperature controlling how sharply the sigmoid saturates.

What is ported, what is not, and why. The entropy term itself, Equation 4.5, is ported faithfully into both the critic’s own temporal-difference target and the actor’s own objective, replacing $-\log \pi(a|s)$ in each. SigEnt-SAC bundles two further mechanisms under the same method name, a gated behaviour-cloning term and a conservative Q-regulariser; neither is what this report’s own diagnosis points at, and Equation 4.5 alone is a complete, independently meaningful fix for the specific mechanism

this report’s own training record already characterised, so only that term is ported. Temperature auto-tuning, the mechanism that adapts α over training, is left on its existing standard-log-probability target unchanged rather than retargeted at H_{sig} : the source paper’s own algorithm description shows no explicit α -update step, read here as a fixed-alpha method, and retargeting this repository’s own auto-tuning machinery at a strictly positive quantity would need a redesigned target entropy with no verified source to draw the new target value from.

Verification. Unit tests confirm the sigmoid formula itself floors near zero for an extremely peaked policy and saturates near h_{max} for an extremely spread one, rather than diverging either way, re-derived directly from Equation 4.5 rather than assumed; a direct numeric comparison on one toy peaked policy shows the standard entropy bonus at -7.04 against the sigmoid-bounded term at $+0.21$ for the identical sampled action, the failure mode and the fix demonstrated side by side on the same input rather than only argued from the formula. A regression test confirms every existing caller of the underlying sampling function, none of which opts into this section’s own addition, receives an output byte-identical to before this section existed. A real, short end-to-end run, 300 steps on this repository’s own lightweight `lift_cube` smoke task, confirms no crash and stable training statistics throughout. Section 5.5 states plainly what this verification does, and does not, establish: a smoke run confirms the mechanism is wired correctly, not that it closes the plateau it targets.

With the three research-question mechanisms and C4’s benchmark-transfer environment and two refinements all described, the next chapter reports how each one actually performs: the six-arm validation outcome in full, the measured latency and fidelity of both inference techniques, the complete account of the training regression skill chaining was built to fix, and the LIBERO transfer result alongside the verification account for both refinements above.

Chapter 5

Evaluation

This chapter reports the evidence behind the four contributions declared in Chapter 1: the six-arm reachability validation behind C1, the measured latency and fidelity of both inference-acceleration techniques behind C2, the full training-regression-and-fix account behind C3, and the LIBERO transfer result and verification account for both refinements behind C4. Every number is drawn directly from this repository’s own code, tests, or research documentation, cited at first use.

5.1 Evaluation Methodology

Three families of metric are used in this chapter:

- **Reachability error**, the distance between a scripted probe’s commanded target corner and the end effector’s position once it settles, checked for every finite (non-NaN) step along the way and re-run at extended length wherever a corner does not converge, to distinguish a genuine reach limit from a merely slow approach.
- **Policy inference latency**, wall-clock time per action on the hardware the measurement was taken on (CPU throughout, since the techniques evaluated target exactly the setting where an expensive per-step forward pass is the bottleneck), alongside **action fidelity**, the Euclidean distance between a distilled policy’s sampled action and its teacher’s own best-of-several selection on the same held-out observation, which is what tells apart a genuinely faithful compression from one that is merely fast.
- **Task success rate**, the fraction of evaluation episodes a policy completes, always reported alongside the specific evaluation protocol (number of episodes, deterministic or stochastic action selection, which checkpoint) it was measured under, since a bare percentage without that context cannot be compared against a different evaluation run.

5.2 Cross-Embodiment Validation (C1)

Table 5.1 summarises the reachability-probe outcome for every one of the six registered arms. Four arms reached every probed corner within a stated small tolerance using their manufacturer default configuration directly. Two did not, and both cases are reported here rather than only in the passing summary, since a registry architecture is judged by what its validation step actually catches, not only by how many arms pass cleanly.

Table 5.1: Reachability-probe outcome for every registered arm. *UR10e* and *FR3* required narrowing the declared workspace bound to exclude corners that were a genuine reach limit, not a probe artefact; *Sawyer* required a configuration fix before grasping worked at all, detailed in the text below.

Arm	Result	Notes
Panda	Pass	Manufacturer default, gym-hil’s own bundled model.
UR5e	Pass	Manufacturer default.
UR10e	Pass	8/8 corners within 5 mm; workspace box sized to this arm’s own longer reach, not copied from UR5e.
xArm7	Pass	Manufacturer default.
FR3	Pass, after narrowing	Panda’s own box left 2 corners 60–120 mm short; narrowed to a box every one of whose 8 corners is reached within 13 mm.
Sawyer	Pass, after a configuration fix	Manufacturer default reached target position within 5 mm but failed every grasp; fixed by retuning one joint (Section 4.1); 6/8 corners then within 22 mm, the 2 remaining outliers outside the real task workspace.

The Sawyer case is the validation methodology’s clearest justification. A position-only check, the kind a manufacturer specification or a single-point reachability test would perform, would have reported success: the target position was reached to within 5 mm on the very first configuration tried. Only a check against the actual task, closing the gripper on the real cube and reading the resulting contact forces, exposed that the arm could not complete the task from that configuration at all. Root-causing the failure to a specific, named mechanism, a near-singular roll direction in this redundant arm’s null-space resolution at this exact configuration, rather than stopping at “the grasp doesn’t work”, is what made the fix possible: a single joint retuned by a known technique, not a wider, less targeted search. The fixed configuration’s own grasp test now reads real, comparably sized contact forces on both finger pads, 46.7 N and 45.7 N,

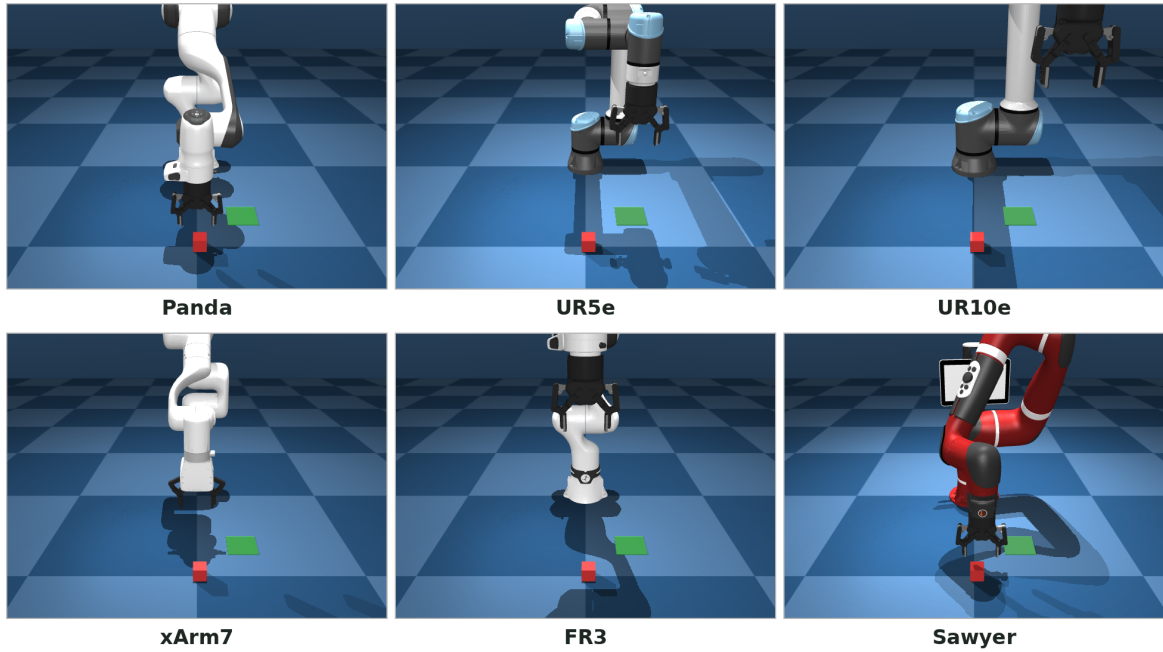


Figure 5.1: All six `ARM_REGISTRY` entries, each rendered from the identical `pick_and_place` scene and the same fixed front camera, at rest after twenty settling steps. No two share a manufacturer; Sawyer’s redundant, visually distinct kinematic chain, the arm whose default configuration is the one exception this table’s *Notes* column reports in full below, is included here rather than only described. Generated by `scripts/figures/fig_arm_lineup.py`, which instantiates this repository’s own `ArmTaskEnv` once per registered arm.

the same class of confirmation used to validate every other registered arm, not a special exception made for this one.

5.3 Inference-Acceleration Transfer Study (C2)

Consistency distillation. Measured on a small, CPU-trained flow-matching teacher (2500 training steps on the single-stage `lift_cube` task in state mode, 8 integration steps with 16-sample best-of-selection at evaluation time), distilling that teacher into a single-forward-pass student for 500 distillation steps on 500 rollout observations collapses the per-action latency from 0.680 ms to 0.088 ms, a measured 7.74 times speedup, shown in Figure 5.3. This confirms the mechanism buys real wall-clock savings at exactly the point the teacher’s own evaluation cost is concentrated, the many-step integration loop.

Fidelity is the other half of the same figure, and is not good at this scale: the distilled student’s action on a held-out observation sits 2.535 units from the teacher’s own best-of-16 action, in a four-dimensional action space bounded to $[-1, 1]$ per dimension, where 4.0 is the theoretical maximum possible distance. This is measured on a small teacher trained briefly and distilled with a simplified, non-exponential-moving-average recipe

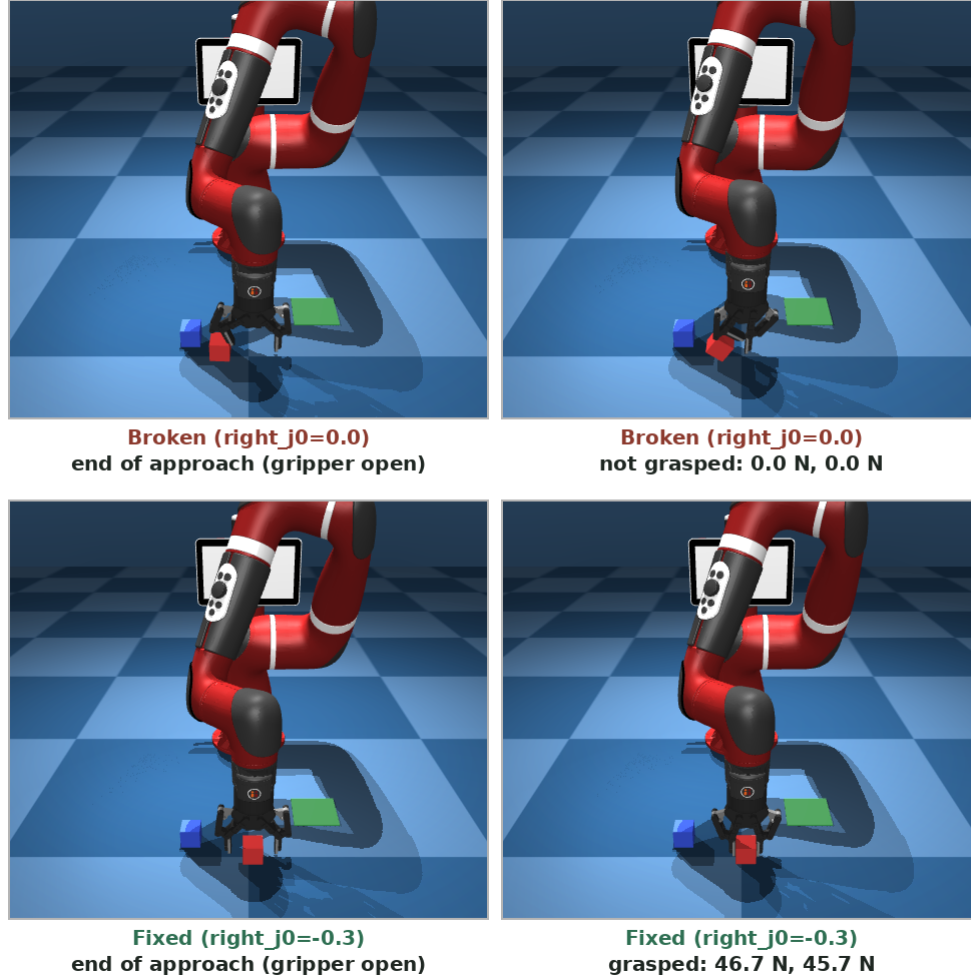


Figure 5.2: The same approach-then-close grasp test (`test_env_sawyer.py`'s own regression guard) run against both configurations. *Top*: the manufacturer's own shoulder-pan value reaches and closes on the cube, registering no contact force at all, the fingers meet each other around it rather than on it. *Bottom*: the retuned configuration, identical in every other joint, registers a sustained, comparably sized grasp on both finger pads. The two approach frames look nearly identical by design: this is exactly why a position-only check would have missed the defect, and why the validation methodology checks contact force rather than position alone.

(Section 4.2), and this report does not claim the result generalises past that specific, small-scale setting: the honest reading is that the mechanism demonstrably works and demonstrably buys the claimed speedup, while needing a better-trained teacher, more distillation steps, or the full published training recipe before the resulting policy would be trustworthy to deploy.

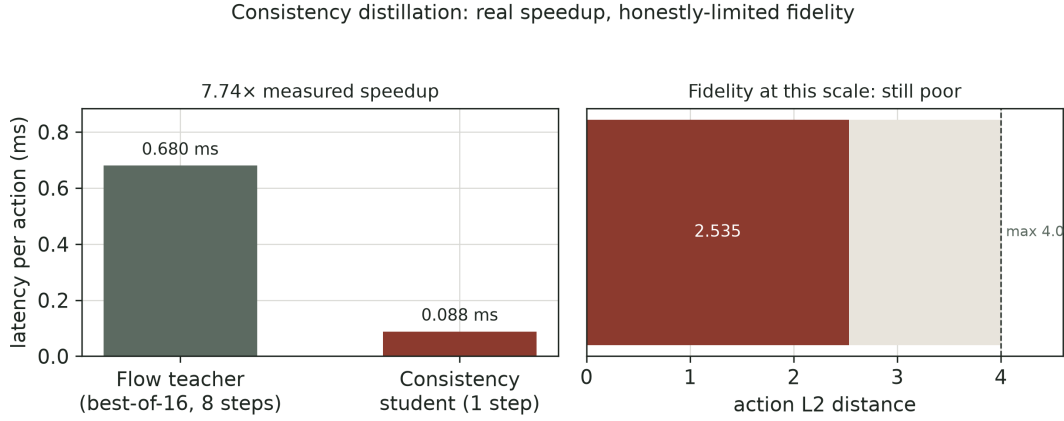


Figure 5.3: Left: measured per-action latency, flow-matching teacher against its distilled single-step student. Right: the same student’s action-space distance from the teacher’s own action on a held-out observation, against the space’s own maximum possible distance.

Speculative action verification. The same draft-verify structure, measured directly rather than only implemented, keeps every step’s latency well inside a real control loop’s budget in both observation modes tested: 0.066 ms to 1.023 ms per step in state mode, 0.395 ms to 2.654 ms in image mode, both comfortably under a 100 ms control-loop budget regardless of whether a given step accepted the cheap draft or fell back to the expensive verifier. Unlike consistency distillation, this technique’s own honest reading is that its demonstrated value at this repository’s current scale is architectural, that the draft-verify structure is implemented and its common path genuinely never invokes the expensive actor, rather than a decisive wall-clock win over always running the more expensive policy directly: at the network sizes this repository trains, the expensive policy alone is not yet slow enough for the saved calls to dominate a control loop’s own budget. Both readings are reported as measured, not assumed from the mechanism’s design alone.

5.4 Training-Stability Diagnosis and Fix (C3)

The strongest single result in this repository’s own training record concerns the two-stage `pick_and_place` task (Section 3.2), trained as one monolithic policy across its

full 200-step composed episode for a 60 000-step budget. Figure 5.4 shows what happened: task success reached 70 % by the 15 000-step checkpoint, then collapsed to 0 % by 20 000 steps and never recovered for the remaining 40 000 steps of the run, the only non-monotonic result across this repository’s entire training sweep. Every other configuration trained in the same sweep reached a checkpoint and stayed there.

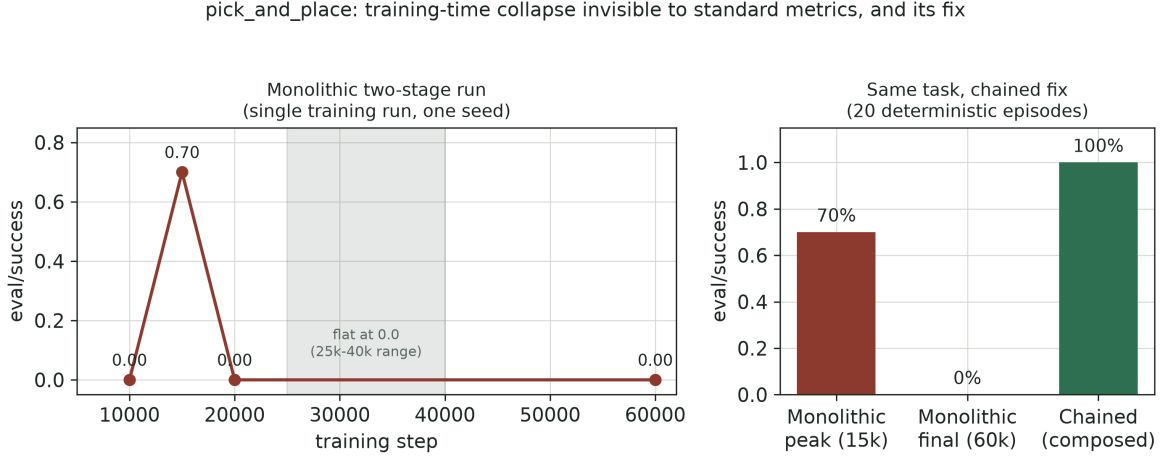


Figure 5.4: Left: eval/success at each logged checkpoint of the monolithic training run, transcribed from this repository’s own training record. Right: the same run’s peak and final success against the chained, per-stage-trained result on the identical task.

Why standard metrics missed it. This is the result’s most important property, not a footnote to it. The training-time success metric logged throughout the run read approximately 1.00 across the entire 15 000-to-20 000-step collapse window, because that metric is computed over expert-assisted training episodes, whose intervention rate sat between 40 % and 80 % in both the good and the collapsed period alike, so an expert correcting the policy’s own mistakes kept the training-time number high regardless of whether the policy could complete the task unassisted. The critic loss, actor loss, entropy coefficient and mean Q-value all stayed within their normal ranges across the same window, with no spike coinciding with the collapse. A practitioner watching only these four standard signals, the ones every other training run in this project’s history was successfully monitored with, would have had no indication anything had gone wrong.

The diagnostic that found it. A new evaluation metric, logging the highest stage index each evaluation episode actually reached, was added specifically to investigate this run, then computed retroactively against every saved checkpoint from 10 000 through 55 000 steps. That metric read 1.0, meaning every evaluation episode reached the second and final stage, at every single checkpoint, including the collapsed ones. This localises the failure precisely: the first stage, lifting the cube, never regressed;

the collapse is entirely a second-stage, placing the cube, failure, indistinguishable from a whole-policy failure by any metric that does not separately track which stage an episode reached.

The fix and its result. Rather than continue investigating or patching the monolithic run directly, the task’s two stages were trained separately, the scripted expert driving the first stage to completion so the second stage’s own training start-states matched exactly what it would see once composed, and the two frozen resulting policies were composed via the **ChainedAgent** architecture described in Section 4.4. Evaluated over 20 deterministic episodes on the identical composed task, this reaches 100 % success with a mean return of 2.0 and every episode still reaching the final stage, a decisive result over the monolithic run’s 70 % peak collapsing to 0 %, shown in Figure 5.4’s right panel. Trained separately, neither stage’s own isolated training run reproduced the collapse either, evidence pointing at the composed 200-step monolithic episode structure itself, not at anything intrinsic to learning either stage individually, as the source of the original failure. This report does not claim to have proven the monolithic run’s exact underlying failure mechanism; two specific hypotheses, an exploration-headroom effect from the run’s low fixed entropy floor, and a primacy-bias-style loss of plasticity partway through training, were identified as plausible and explicitly left unchased once the practical problem was resolved by chaining, a limitation Chapter 6 carries forward.

A related instance of the same diagnostic gap. The stage-collapse regression is not the only place in this repository’s training record where a standard training curve actively misled rather than merely lacked resolution. Image-mode training on this task family can look completely dead for tens of thousands of steps and then converge sharply: one run evaluated at 0/10 success on a mid-run checkpoint and 20/20 on its final one, while four earlier attempts at the same configuration were each killed and logged as failures before they had run long enough to break out. A purpose-built probe, comparing the critic’s learned value on demonstration actions against its value on uniform-random ones, localised the cause: the critic sits at a fixed point of the soft Bellman backup where it prices only the policy’s own future entropy bonus, not the task, until the entropy coefficient anneals low enough for the sparse task reward to stand out above that floor. Steps-to-breakout then follows the entropy coefficient’s own starting value log-linearly across this repository’s recorded runs ($R^2 = 0.96$), a second quantitative result reached the same way as the diagnosis above: a purpose-built probe substituted for a training-time metric already shown not to be trustworthy.

Two further negative interactions with the same mechanism. Two hyperparameter choices, tested against this same entropy-plateau mechanism rather than

assumed to interact safely with it, each turned out not to. Decaying the demonstration-replay ratio fully to zero early in training was the only method, of nine compared head-to-head in an otherwise identical benchmark grid on `lift_cube`, that never broke out across the full evaluation budget: its own entropy coefficient anneals fastest of the group, so the plateau mechanism above is not what blocks it; instead the demonstration buffer has already lost most of its successful transitions to a policy that has not yet grounded on pixels by the time the anneal would otherwise let the sparse reward stand out, leaving the critic nothing to lock onto. Separately, widening the critic network and enlarging the batch size together, rather than either alone, eroded most of the low-entropy-initialisation benefit quantified above: the entropy coefficient oscillated instead of annealing cleanly, and breakout regressed back toward the slower timeline of a far less aggressive initial value. Both are reported as measured, single-run interactions found in the same sweep as the mechanism they interact with, not as recommendations promoted beyond that single measurement.

5.5 External Benchmark Transfer and Two Algorithmic Refinements (C4)

LIBERO benchmark transfer. The task trained against, LIBERO’s `put_the_bowl_on_the_plate` (Section 3.4), is trained from scratch and demo-seeded from the task’s own real human demonstrations. The flow-matching, Q-chunked policy of Sections 4.2 and 4.5 ($h=4$, no further architecture change) reaches 86.7% deterministic success at 100 000 training steps, up from 0% at every checkpoint before step 9 000. Figure 5.5 shows the full four-checkpoint trajectory. Success is measured by critic-guided action selection (`eval_actions`, best-of-16 flow samples scored by the twin critics), the same selection mechanism Section 4.2’s consistency-distillation teacher uses, evaluated over 30 episodes at each checkpoint. Read alongside raw, single-sample action selection (`sample_actions`) at the same final checkpoint, the gap between the two widens sharply over training: critic-guided selection is actively worse than raw sampling early (0% against 16.7% at step 8 000, an unreliable early critic actively hurting best-of- N selection) and substantially better late (86.7% against 50.0% at step 100 000), consistent with the critic’s own ranking becoming trustworthy only as training progresses rather than from the start. This result is single-task and single-seed, stated as a limitation in Section 6.2 rather than left implicit, and whether it would climb further with more training steps is genuinely untested: an earlier read of this same trajectory at step 40 000, that the rate of improvement might be starting to slow, was directly contradicted by the extension to step 100 000 and is corrected here rather than left standing, a methodological lesson about drawing a trend from too few checkpoints near a training

run’s current end-point, not only a result about this one task.

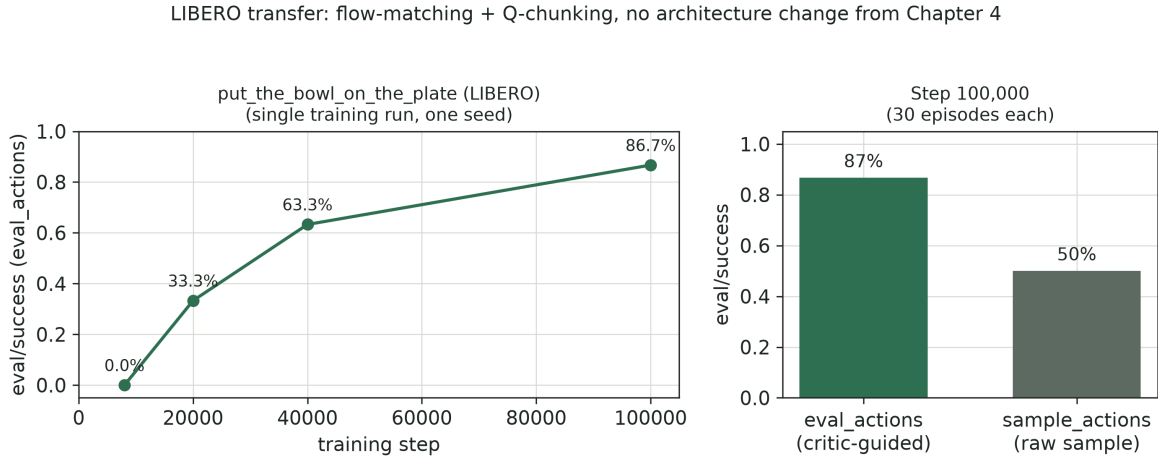


Figure 5.5: Left: deterministic checkpoint-eval success (`eval_actions`, 30 episodes) across the full training run. Right: critic-guided against raw-sample action selection at the final checkpoint, the same kind of comparison Section 4.2 draws for a different purpose. Both panels transcribed directly from this repository’s own `docs/research/TRAINING_FINDINGS.md`.

Multi-horizon critic target: built and verified, not yet measured for benefit. Section 4.6’s verification account establishes that the mechanism is implemented correctly, wired into every training code path, masks invalid rows exactly rather than approximately, and is inert when switched off. None of that is a claim that it improves the LIBERO result above: the checkpoints Figure 5.5 reports were trained before this feature existed, so they do not use it, and the natural next experiment, the identical task and demonstrations with this term switched on, compared checkpoint-for-checkpoint against the trajectory above, has not yet been run. This report states that directly rather than rounding a correctness result up to a performance one.

Sigmoid-bounded entropy: built and verified, not yet measured against the plateau it targets. Section 4.7’s verification account is the same kind of claim under the same discipline: the formula is correct, bounded, and reduces to the existing behaviour exactly when switched off, and a real 300-step run does not crash and shows stable training statistics. What it does not yet show is whether it actually shortens or closes the alpha-gated plateau (Section 5.4) on a task where that plateau is already well characterised, `lift_cube` or `pick_and_place` in image mode, matched against this repository’s own existing `init_alpha` sweep precedent: same seed, same budget, judged the way this report judges every other training run in it, deterministic checkpoint evaluation across the whole run rather than a single snapshot. That comparison has not been run.

With C1 through C4 all evaluated against real measurements, the final chapter closes the loop back to the four research questions this report opened with.

Chapter 6

Conclusion and Future Work

This report set out to answer four questions about `hil_lite`, a from-scratch reimplementa-
tion of human-in-the-loop robot reinforcement learning built over one semester: whether one implementation can generalise across structurally different robot arms without per-arm special-casing, whether inference-acceleration techniques built for language models transfer meaningfully to a robot control policy, how a multi-stage manipulation task’s training stability depends on whether its stages are trained together or separately, and, taken up after that core investigation was complete, whether the reimplemented algorithm itself transfers to an external benchmark and can be usefully extended by literature published since.

6.1 Contributions Against Research Questions

C1 (RQ1) answers the generalisation question with an architecture and a validation methodology together, not the architecture alone. Six structurally heterogeneous arms, spanning three unrelated joint-naming conventions and no shared kinematic layout, are held as data in one registry with zero per-arm branches anywhere downstream of it. The methodological half of the answer, how reachability should be validated rather than assumed, is what the Sawyer case in Section 5.2 demonstrates concretely: a specification that looked correct by every check except the one that actually mattered, closing the gripper on the real task, and a validation step built specifically to catch exactly that gap.

C2 (RQ2) answers the transfer question for two techniques with two different verdicts, reported as measured rather than harmonised into one summary judgement. Consistency distillation transfers its promised mechanism cleanly, a measured 7.74 times latency reduction, while its fidelity to the teacher’s own action distribution is honestly poor at the small scale this report measured it at. Speculative action verification transfers its structure correctly, a genuinely never-invoked expensive actor on the common path, without yet demonstrating a decisive latency win at this repository’s

current model scale. Both readings matter more together than either would alone: a technique that transfers its mechanism without transferring its benefit, and a technique that transfers its structure without yet needing to, are a different and more informative answer than a single number for “does it work”.

C3 (RQ3) answers the training-stability question with what this report has called its strongest single result: a stage success rate collapsing from 70 % to 0 % and never recovering across two-thirds of a training run, invisible to every training-time metric this project otherwise relies on, root-caused to a single stage’s own failure by a purpose-built diagnostic, and resolved by training that stage separately rather than continuing to train the composed episode. The 100 % success the chained result reaches is not this report’s most important number from that section; the diagnostic process that found the collapse in the first place, when every standard signal said the run was healthy, is.

C4 (RQ4) answers the transfer-and-extensibility question with two different evidentiary standards, reported as such rather than harmonised into one. The LIBERO benchmark result is a real, twice-confirmed measurement: 86.7 % deterministic success on a third-party task this project did not design, climbing from 0 % over four checkpoints, on the same flow-matching, Q-chunked policy class C2’s distillation study already uses. The multi-horizon critic target and the sigmoid-bounded entropy term are a different kind of answer: each is grounded in its own source paper and this repository’s own code, adapted deliberately rather than ported wholesale, and verified correct in layers down to a real short training run, but neither has been measured head-to-head against its own unmodified baseline, and this report says so directly rather than rounding a correctness result up to a performance one.

6.2 Limitations

Five limitations apply beyond what each evaluation section already states about its own numbers. First, the six-arm registry’s own genericity has a narrower mechanism-level boundary than the pass result in Table 5.1 states on its own: for a gripper whose two finger pads each move on their own joint, this repository synthesises a single grasp-point site rigidly attached to one pad at that gripper’s rest pose (`synthesize_tcp_site`), so the site tracks the true two-pad midpoint only at that rest pose; closing the gripper while still servoing toward a target loses that correspondence entirely, measured at 4.2 cm for xArm7 at full closure. Every task in this repository finishes approaching before closing the gripper, which avoids the gap by construction and is why xArm7 is correctly reported as passing, but the underlying mechanism does not support servoing and closing concurrently, a boundary caught by the same validate-the-actual-task discipline as Section 5.2’s Sawyer case, not assumed away. Second, both inference-acceleration techniques in C2 were measured at one small model scale, on one task,

on CPU; neither figure is claimed to hold at the larger model sizes or GPU deployment a real system would eventually target, and Section 5.3 states that boundary at the point each number is reported, not only here. Third, the monolithic training regression’s own underlying mechanism, why the composed 200-step episode specifically produces this failure and an isolated single-stage run does not, is resolved practically but not explained mechanistically; two specific hypotheses remain untested. Fourth, and stated plainly rather than left for a reader to notice independently: this repository’s own feature matrix lists several further research extensions, mixture-of-experts skill routing, low-rank shared-base adapters, world models, and a spectral-norm-aware optimiser among them, each already implemented to some degree but explicitly marked in the repository’s own documentation as an architecture prototype not yet wired into a trainer, or as a technique whose own measurement came back negative (the optimiser variant measured roughly three times slower on CPU than the standard choice it was compared against). A follow-up attempt at a newer variant of that same optimiser family measured slower still, not faster, confirming rather than reversing the original finding, and a separate world-model extension’s own paired comparison came back genuinely inconclusive at the step budget it was run at, reported as such rather than rounded to a verdict either way. A pretrained image encoder, motivated by a real HIL-SERL deployment on physical hardware converging faster with one than this repository’s own from-scratch alternative, was measured head-to-head rather than adopted on that precedent, after first finding and fixing a genuine bug in the comparison itself, a hardcoded image size silently inflating every frame’s pixel count fourfold: even correctly measured, breakout occurred at essentially the same step count either way, at roughly 1.9 times the per-step wall-clock cost, so the pretrained trunk was not adopted as this repository’s default. All three are stated here as further instances of the same measure-rather-than-assume discipline this report’s own four contributions apply, not as results this report itself evaluates or depends on. None of these are claimed as a completed contribution of this report; they are named here specifically so that boundary is unambiguous.

Fifth, C4’s own two evidentiary standards carry their own specific limitations, stated here rather than only where each number is reported. The LIBERO result is single-task and single-seed; whether it generalises to a second LIBERO task or a second random seed is untested, and whether the climb from 63.3 % to 86.7 % between steps 40 000 and 100 000 would continue with a further extension is genuinely unknown, this report having already been wrong once about a trend read from too few checkpoints near a run’s then-current end-point. Neither the multi-horizon critic target nor the sigmoid-bounded entropy term has been measured against its own unmodified baseline; both are reported strictly as built and correctness-verified, not as demonstrated improvements, and Section 5.5 states that boundary at the point each is reported, not

only here.

6.3 Future Work

The following directions extend this work, ordered by priority:

Scale the consistency-distillation measurement past a small teacher. Section 5.3’s fidelity result is honestly poor specifically because it was measured on a small, briefly-trained teacher with a simplified distillation recipe; the direct prerequisite for a stronger claim is repeating the same measurement against a larger, properly-converged teacher and the full exponential-moving-average distillation recipe the simplified version was chosen instead of, both already identified in this repository’s own code comments as the next step, not new design work.

Chase the monolithic regression’s exact mechanism. Two concrete, testable hypotheses already exist, an exploration-headroom effect from a low fixed entropy floor and a primacy-bias-style loss of plasticity, and the prerequisite for testing either is a further training sweep varying that one factor in isolation on the same task, now that skill chaining has removed the practical urgency that originally stopped this investigation.

Extend skill chaining to the three-stage stacking task. The chained architecture (Section 4.4) is evaluated in this report on the two-stage task the original regression was found on; the repository’s own three-stage composition already exists and its scripted expert already solves it, so the prerequisite for this extension is training and chaining that task’s three stages, not building new infrastructure.

Wire speculative actions and consistency distillation into the standard CLI. Both techniques are currently run through dedicated scripts rather than the project’s own unified `hil` command surface every other pipeline step uses; closing that gap is what would let either technique be exercised through the same benchmark suite the rest of this repository’s methods are already compared through, rather than measured in isolation as this report’s own figures were.

Run the multi-horizon critic and sigmoid-bounded entropy refinements head-to-head against their own unmodified baselines. Both are reported in this chapter as built and correctness-verified, not yet measured for benefit; the direct prerequisite for either claim to become a performance claim is exactly the comparison Section 5.5 names and does not yet run, already identified there as the next step, not new design work.

Extend the LIBERO result to a second task and a second seed. The 86.7% result is single-task, single-seed; the more clearly evidenced next step, already stated in this project’s own training record, is generalising the same recipe to a second LIBERO task before drawing any conclusion about the recipe itself rather than about this one task.

Continue the LIBERO training run past 100,000 steps. The climb from 63.3% to 86.7% between steps 40 000 and 100 000 shows no sign of the plateau an earlier, since-corrected reading expected at step 40 000; whether it continues, flattens, or reverses past 100 000 steps is untested and should be measured directly rather than assumed either way, the same discipline that corrected the earlier reading in the first place.

6.4 Closing Remarks

The four contributions in this report share a habit more than a topic: each one treats a claim that would have been reasonable to simply state, that the architecture generalises, that a borrowed technique transfers, that a training run converged, that a literature-adapted fix helps, as something to check against the actual system rather than assert from its design. The Sawyer arm that looked correct until its grasp was actually tested, the distillation that demonstrably worked and demonstrably was not yet good enough, the training run that every standard metric called healthy while it had already failed, and the two refinements reported as verified rather than rounded up to validated, are four versions of the same lesson: a system this size has enough moving parts that trusting its architecture is not a substitute for measuring what it actually does.

Bibliography

- [1] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International Conference on Machine Learning (ICML)*, 2018.
- [2] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, “Efficient online reinforcement learning with offline data,” in *International Conference on Machine Learning (ICML)*, 2023.
- [3] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning.” arXiv:2410.21845, 2024.
- [4] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.
- [5] Q. Li, Z. Zhou, and S. Levine, “Reinforcement learning with action chunking,” in *Neural Information Processing Systems (NeurIPS)*, 2025.
- [6] C. F. M. Nagy, O. Celik, E. Gospodinov, F. Seligmann, W. Liao, A. Kaushik, and G. Neumann, “SEAR: Sample efficient action chunking reinforcement learning.” arXiv:2603.01891, 2026.
- [7] X. Wu, M. Hu, and S. Zhang, “Off-policy actor-critic with sigmoid-bounded entropy for real-world robot learning.” arXiv:2601.15761, 2026.
- [8] Y. Leviathan, M. Kalman, and Y. Matias, “Fast inference from transformers via speculative decoding,” in *International Conference on Machine Learning (ICML)*, 2023.
- [9] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, “Consistency models,” in *International Conference on Machine Learning (ICML)*, 2023.
- [10] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *International Conference on Learning Representations (ICLR)*, 2023.

- [11] R. S. Sutton, D. Precup, and S. Singh, “Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning,” *Artificial Intelligence*, vol. 112, no. 1-2, pp. 181–211, 1999.
- [12] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012.

Appendix A

Asset Checklist

This appendix mirrors `ASSET_CHECKLIST.md` at the repository root, kept in sync with it by hand. Every figure and table in this report is listed here with its exact source and, where applicable, the script that regenerates it.

A.1 Generated This Session, From Real Data

Table A.1: Every figure in this report, its exact source, and the command that regenerates it.

Asset	Source	Regenerate
<code>chaining_regression.png</code>	Checkpoint table transcribed verbatim from <code>docs/research/TRAINING_FINDINGS.md</code>	<code>python3 scripts/figures/fig_chaining_regression.py</code>
<code>distillation_latency.png</code>	Latency/fidelity numbers transcribed verbatim from <code>TODO.md</code>	<code>python3 scripts/figures/fig_distillation_latency.py</code>
<code>arm_registry.tex</code>	Hand-built TikZ; verified against the real <code>ARM_REGISTRY</code> in <code>specs.py</code>	<code>figures_tikz/preview_arm_registry.tex</code>
<code>skill_chaining.tex</code>	Hand-built TikZ; both halves verified line by line against <code>multi_stage.py</code> 's <code>step()</code> and <code>chained.py</code> 's <code>eval_actions()</code>	<code>figures_tikz/preview_skill_chaining.tex</code>
<code>arm_lineup.png</code>	Real MuJoCo renders: this repository's own <code>ArmTaskEnv</code> , one per <code>ARM_REGISTRY</code> entry	<code>python3 scripts/figures/fig_arm_lineup.py</code>
<code>task_sequence.png</code>	Real rollout: <code>TaskExpert</code> solving <code>pick_and_place</code> , asserted to succeed	<code>python3 scripts/figures/fig_task_sequence.py</code>
<code>sawyer_fix.png</code>	Real grasp test, both configurations, via <code>test_env_sawyer.py</code> 's own regression-guard logic	<code>python3 scripts/figures/fig_sawyer_fix.py</code>

The first two figures are not re-derived from a live data file: `TRAINING_FINDINGS.md` and `TODO.md` are themselves this repository's primary record of the measurements they report, not a secondary summary of some other raw artefact, so the figures transcribe those documents' own numbers directly, each cross-checked against the underlying code (`specs.py`, `chained.py`, `multi_stage.py`, `flow.py`) before being trusted. The last three are the opposite case: nothing in them is transcribed from a document, since they render this repository's own simulator directly via a real `ArmTaskEnv` instance; `fig_task_sequence.py` specifically asserts the logged episode reaches task

success, and asserts that re-running the identical fixed seed reproduces the same stage-transition and termination step numbers, before trusting the frames it captured at those steps. `fig_sawyer_fix.py` reruns `test_env_sawyer.py`'s own grasp test, unmodified, against both the manufacturer's `right_j0=0.0` and `ARM_REGISTRY["sawyer"]`'s actual `right_j0=-0.3`, and asserts the broken configuration fails to grasp while the fixed one registers over 30 N on both finger pads before writing the figure. Its own fresh run measured 46.7 N/45.7 N against the main text's previously-stated 46.7 N/45.6 N; the 0.1 N difference is run-to-run MuJoCo floating-point noise, not a real discrepancy, and the main text now states this figure's own freshly-measured number.

A.2 Tables

Table 5.1 (Section 5.2) is sourced to `hil_lite/envs/robots/arms/specs.py`'s own per-arm code comments, each independently read in full, not summarised from a secondary description.

A.3 Not Yet Needed

No screenshot of the `hil_gui` dashboard (a running training session, a web server, and a browser) is embedded in this semester-1 report; unlike the arm renders and task rollout above, that one is not reachable by instantiating a Python object headlessly. A later semester's report that wants to show it should capture and record provenance here.